

TRACE / Llama-3.2-1B-Instruct Log- t VAMP

Experiment Specification

Revision 2 — Core-Space Merging, Persistent Artifact Cache, and 2×4090 RunPod Execution

1. Purpose

Implement and evaluate log- t temporal-consolidation VAMP on the **TRACE continual-learning benchmark** using **Llama-3.2-1B-Instruct**.

The main research question is:

Can a continual learner approach joint-IID adaptation quality while storing only $O(\log T)$ live parameter memories and performing bounded inference-time addressing?

The experiment must separately determine:

1. whether the log- t hierarchy itself preserves knowledge;
2. whether cheap parameter-space LoRA merging can replace expensive replay consolidation;
3. whether a small amount of repair replay closes any remaining merge gap;
4. whether failures come from consolidation or from inference-time addressing;
5. how close each method comes to a joint-IID LoRA trained on the same total data.

This follows the VAMP addressed-parameter-memory view in which historical parameter adaptations remain separately recoverable instead of all being forced into one continually overwritten model.

2. Mandatory implementation recommendation

Build the 40 level-zero LoRAs exactly once, save every one permanently, and treat them as immutable experimental inputs. Never retrain them merely because a merge algorithm, merge hyperparameter, replay fraction, router, or reporting method changes.

Likewise:

- never delete intermediate merge artifacts;
- never overwrite an adapter with a different merge policy;
- identify every derived adapter by its source nodes plus a complete policy/configuration hash;
- keep merge-specific intermediate representations such as singular spectra and Core-Space bases;
- make consolidation and evaluation independently rerunnable from stored artifacts.

The experiment should therefore be decomposed into:

expensive leaf training once

followed by arbitrarily many cheap:

merge policy → repair policy → routing/evaluation

reruns.

This is an architectural requirement, not an optional optimization.

A later experiment such as:

```
Core+TSV
repair_fraction = 0.10
rank = 12
different Core scaling
different routing method
```

must **not** require retraining any level-zero LoRA.

3. Benchmark protocol

Use the TRACE **500-training-example-per-task** variant used by the current TreeLoRA implementation:

LLM-CL-Benchmark_500

and the canonical sequence:

```
C-STANCE
FOMC
MeetingBank
Py150
ScienceQA
NumGLUE-cm
NumGLUE-ds
20Minuten
```

The current TreeLoRA TRACE script uses this dataset variant and sequence.

Use:

```
meta-llama/Llama-3.2-1B-Instruct
```

with the exact model and tokenizer revisions recorded in the run manifest.

Base weights remain frozen for all LoRA methods.

Use BF16. Do not use QLoRA in the headline experiment.

4. TRACE formatting must match the reference implementation

Do not apply Meta's chat template.

Replicate TRACE's existing causal-LM formatting:

```
prompt + answer
```

with:

```
max_prompt_len: 1024
max_answer_len: 512
padding_side: left
truncation_side: left
```

Loss is computed **only on answer tokens**. Prompt tokens are masked to `-100` in the target labels. The current TRACE/TreeLoRA data collator implements these semantics directly.

Do not silently change this.

5. Reference LoRA and training configuration

Use:

```
rank: 8
alpha: 32
```

```
dropout: 0.1
bias: none

target_modules:
- q_proj
- v_proj
```

The reference Llama mapping applies LoRA to `q_proj` and `v_proj`.

Therefore the LoRA scale is:

$$s = \frac{\alpha}{r} = 4.$$

All merge implementations must incorporate this scaling correctly.

Match the executed reference optimization behavior:

```
optimizer: Adam
betas: [0.9, 0.95]
learning_rate: 1.0e-4
weight_decay: 0
scheduler: constant
warmup_steps: 0

micro_batch_size: 1
gradient_accumulation_steps: 8
effective_batch_size: 8
```

Use task epoch counts:

C-STANCE	5
FOMC	3
MeetingBank	7
Py150	5
ScienceQA	3
NumGLUE-cm	5
NumGLUE-ds	5
20Minuten	7

These are the settings in the current TRACE LoRA script.

6. Continual stream: 40 arrivals

Each 500-example task is deterministically split into five 100-example arrivals.

Thus:

$$T = 8 \times 5 = 40.$$

Preserve blocked task order:

```
C-STANCE 1..5  
FOMC 1..5  
...  
20Minuten 1..5
```

Use seed:

```
1234
```

for dataset partitioning and the first full run.

Create and hash a permanent dataset manifest containing every example ID and arrival assignment before training begins.

No later merge-policy rerun may alter these leaves.

7. Level-zero training

For each arrival D_t :

1. load the common frozen base;
2. instantiate a fresh zero-effect rank-8 LoRA;
3. train only that LoRA on the 100 examples in D_t ;
4. use the epoch count of the current TRACE task;
5. save the resulting LoRA as an immutable artifact.

Each leaf is base-relative:

$$\theta_t = \theta_0 + \Delta_t.$$

Never initialize leaf t from leaf $t - 1$.

The 40 leaves collectively receive exactly:

$$500(5 + 3 + 7 + 5 + 3 + 5 + 5 + 7) = 20,000$$

example presentations.

8. Temporal hierarchy

Use the existing VAMP temporal rule:

keep at most two live nodes at each level; when a third appears, merge the two oldest equal-level nodes and carry the result upward.

This is the policy already specified for the TinyWorlds temporal-consolidation experiment.

A level- j node represents:

$$2^j \times 100$$

examples.

After 40 arrivals, the exact live hierarchy must be:

L0: [39] [40]
L1: [37-38]
L2: [33-36]
L3: [17-24] [25-32]
L4: [1-16]

There are therefore:

$$7$$

live adapters and:

$$33$$

historical merge events.

Including the base gives eight exhaustive routing candidates.

Unit tests must verify this exact topology.

9. Merge methods

The experiment must support merge methods as interchangeable policies.

The initial comparison includes two primary merge families.

M1 — Weighted sum followed by optimal rank-8 truncation

Name:

svd_mean_r8

For children:

$$\Delta_A = s_A B_A A_A, \quad \Delta_B = s_B B_B A_B,$$

form:

$$\Delta = w_A \Delta_A + w_B \Delta_B,$$

where weights are proportional to represented example count.

Equal-level merges therefore normally have:

$$w_A = w_B = \frac{1}{2}.$$

Represent the exact combined update using stacked low-rank factors, then compute the best rank-8 approximation through compact QR/SVD.

Never average A and B factors separately.

Do all QR/SVD operations in FP32.

Store the rank-8 parent as an ordinary PEFT-compatible LoRA.

10. M2 — Core Space + TSV + rank-8 output

Name:

core_tsv_r8

This is the second primary merge method.

The Core Space Merging framework aligns independently trained LoRA subspaces into a shared compact basis and then performs the actual merge in that basis. Its official implementation supports Task Arithmetic, TIES, DARE, DARE-TIES, TSV, and CART.

Use **Task Singular Vectors (TSV)** as the nonlinear merge operation.

This is the best applicable method from the Core Space paper's language experiments: Core Space + TSV reached **94.16% normalized average accuracy** on its Llama-3 8B NLI experiment.

This is not evidence that it will necessarily dominate on TRACE; that is what we are testing.

10.1 Pairwise Core-Space construction

For each adapted matrix and two children with low-rank factors:

$$\Delta_i = s_i B_i A_i,$$

construct a shared right basis from the stacked A_i matrices and a shared left basis from the stacked B_i matrices, following the Core Space algorithm.

Conceptually:

$$V_{\text{ref}} = \text{rightbasis} \begin{pmatrix} A_A & A_B \end{pmatrix},$$

$$U_{\text{ref}} = \text{leftbasis} \begin{pmatrix} B_A & B_B \end{pmatrix}.$$

Each child becomes a compact aligned core matrix:

$$M_i = U_{\text{ref}}^\top \Delta_i V_{\text{ref}}^\top.$$

The official implementation constructs these aligned core matrices directly from the LoRA factors.

Apply TSV to:

$$M_A, M_B$$

to obtain:

$$M_*.$$

10.2 Do not materialize a full dense parent merely to recompress it

The official generic implementation reconstructs:

$$\Delta_* = U_{\text{ref}} M_* V_{\text{ref}},$$

but VAMP should preserve low-rank efficiency.

Instead SVD the small core:

$$M_* = P \Sigma Q^\top.$$

Because the reference bases are orthonormal, a rank-8 truncation can be constructed directly:

$$\Delta_*^{(8)} = (U_{\text{ref}} P_8) \Sigma_8 (Q_8^\top V_{\text{ref}}).$$

Factor this directly into the parent's rank-8 LoRA B_P, A_P , accounting for the parent's α/r scale.

This gives:

Core Space

- TSV nonlinear interference reduction
- optimal rank-8 truncation
- ordinary rank-8 LoRA artifact

without constructing a full model-sized dense delta.

10.3 Diagnostic uncompressed Core result

For selected merge events, also retain the pre-rank-8 Core+TSV result conceptually at rank up to 16.

Name:

core_tsv_precompress

It is a diagnostic only.

Compare:

Core+TSV before rank-8 compression
Core+TSV after rank-8 compression

to determine whether any observed damage came from TSV itself or from forcing the result back to rank 8.

Do not let rank double recursively in the deployed VAMP hierarchy.

11. Core-Space scaling hyperparameter

The official Core Space language evaluation code validation-searches the merge scaling coefficient over:

0.1, 0.3, 0.5, 0.7, 0.9.

Support the same grid.

For the first preregistered run:

```
core_scale = 0.3
```

is the fixed default.

Because all leaves and merge inputs are retained, separately permit a whole-tree validation sweep:

```
0.1  
0.3  
0.5  
0.7  
0.9
```

without retraining leaves.

The selected coefficient must be chosen using TRACE validation data, never test data.

Record fixed-default results separately from validation-selected results.

Do not tune a different coefficient against the test set for every merge.

12. Repair replay must be parameterized

Repair is an optional stage after either merge algorithm:

```
children  
→ merge  
→ rank-8 parent  
→ optional repair
```

Support:

```
repair_fraction
```

as a first-class merge-policy parameter.

For a node representing n examples, retain a deterministic replay reservoir of approximately:

$$k = \lceil fn \rceil$$

samples where:

$$f = \text{repair_fraction}.$$

Assign every training example a permanent deterministic hash priority.

The node's reservoir consists of the lowest-priority eligible examples in its represented interval.

When two nodes merge:

1. construct the parameter-space parent;
2. train it for one repair epoch on the union of the two child reservoirs;
3. derive the parent's reservoir deterministically.

Primary configuration:

```
repair_fraction = 0.05
```

Support cheap reruns at least at:

```
0
0.02
0.05
0.10
0.25
```

without leaf retraining.

For the 40-arrival tree, all parent intervals summed over all 33 merges represent 12,200 leaf-arrival units of 100 examples each before applying the replay fraction. Consequently, one repair epoch at 5% corresponds to roughly **610 replay example presentations**, and 10% to roughly **1,220**, modulo rounding.

This is small relative to the 20,000 original leaf presentations.

13. Repair optimizer

Initialize repair from the merged parent.

Use:

```
epochs: 1
optimizer: Adam
betas: [0.9, 0.95]
learning_rate: 5.0e-5
weight_decay: 0
scheduler: constant

micro_batch_size: 1
gradient_accumulation_steps: 8
```

Do not increase LoRA rank during repair.

Record repair work independently from initial leaf work.

14. Primary VAMP matrix

The first full seed must include:

```
svd_mean_r8
svd_mean_r8 + 5% repair

core_tsv_r8
core_tsv_r8 + 5% repair
```

Thus the primary experiment separates two questions:

Does nonlinear Core+TSV beat simple optimal low-rank averaging?

Does either method substantially benefit from a tiny replay repair budget?

Additional repair fractions and Core scaling coefficients are rerun from the persistent artifact cache.

15. Persistent artifact architecture

All expensive and derived artifacts must live in a content-addressed experiment store.

Nothing is overwritten.

Recommended persistent layout:

```
/workspace/vamp-trace/  
  cache/  
    huggingface/  
    trace/  
  
  runs/  
    <run_contract_hash>/  
      manifests/  
        dataset.json  
        model.json  
        leaves.jsonl  
        policies.jsonl  
        jobs.jsonl  
  
      leaves/  
        <leaf_id>/  
          adapter.safetensors  
          adapter_config.json  
          train_metrics.json  
          source_ids.json  
          sha256.txt  
  
      merge_cache/  
        <childA>__<childB>/  
          stacked_factors/  
          svd/  
          core_space/  
          spectra/  
          validation_cache/  
  
      derived/  
        <policy_hash>/  
          nodes/  
            <node_id>/  
              adapter.safetensors  
              adapter_config.json  
              lineage.json  
              merge_metrics.json
```

```
repair_metrics.json
```

```
baselines/  
  sequential/  
  joint_iid/  
  taskwise/  
  
evaluations/  
  <policy_hash>/  
  <router_hash>/  
  
reports/  
logs/  
checkpoints/  
state/
```

The same pair of children may therefore support:

```
svd_mean  
Core+TSV  
different rank  
different Core scaling  
different repair fraction  
different repair optimizer
```

without touching leaf training.

16. Separate experimental retention from algorithmic memory accounting

For reproducibility, the experiment may retain:

```
all leaves  
all parents  
all source IDs  
all TRACE data  
all merge caches  
all evaluation outputs
```

forever.

This does **not** mean VAMP receives all of this as live deployment memory.

Reports must distinguish:

Experimental archive

Everything retained so that experiments can be rerun.

Algorithmically live state

Only:

```
current  $O(\log T)$  active LoRAs  
current router metadata  
replay buffer actually allowed by the selected policy
```

Use the latter when claiming memory complexity.

17. Routing

For each hierarchy, evaluate the same routing methods so merge quality and routing quality remain separable.

Task-free router 1: prompt NLL

Score the observed prompt under:

```
base  
+  
all active temporal LoRAs
```

and choose the lowest length-normalized prompt NLL.

Never inspect answer tokens.

Task-free router 2: frozen-base prompt centroid

Embed the prompt using the frozen base.

Each temporal node stores a weighted centroid of its represented training-prompt embeddings.

Choose the live node with highest cosine similarity.

Diagnostic task-aware selector

For each TRACE task, use validation data and task identity to select the best active candidate.

Report separately.

Answer oracle

Select the best candidate using the actual target answer.

This is diagnostic only.

It answers whether poor performance was caused by:

```
bad memory  
or  
bad addressing
```

18. Baselines

Required:

```
frozen_base  
seq_lora_reference  
joint_iid_lora  
taskwise_lora
```

Also implement:

```
seq_lora_40
```

using the same 40-arrival stream.

Do not drop this merely because the run takes longer than expected.

The experiment is now governed by a resumable 24-hour pause, not by deleting conditions to hit an arbitrary wall-clock budget.

19. Joint-IID baseline

Train a single fresh rank-8 LoRA on the pooled eight-task data.

Match the total presentation weighting implied by the standard task epoch schedule:

20,000

example presentations.

Globally shuffle the resulting joint stream.

The central quantity is:

$$G_{\text{IID}} = OP_{\text{joint}} - OP_{\text{VAMP}}.$$

20. TRACE task metrics

Use the existing TRACE mappings:

C-STANCE	accuracy
FOMC	accuracy
MeetingBank	ROUGE-L
Py150	similarity
ScienceQA	accuracy
NumGLUE-cm	accuracy
NumGLUE-ds	accuracy
20Minuten	SARI

The current reference collector uses this mapping.

Use reference decoding:

```
temperature: 0.1
do_sample: true
max_new_tokens: 512
seed: 1234
```

The current TRACE inference implementation uses stochastic generation at temperature 0.1.

21. Evaluation schedule

Save an immutable hierarchy snapshot after arrivals:

```
5
10
15
20
25
30
35
40
```

Evaluate all tasks encountered to date.

Construct the standard triangular score matrix:

$$A_{i,j}.$$

Do this for:

```
seq_lora_reference
seq_lora_40

svd_mean_r8
svd_mean_r8_repair05

core_tsv_r8
core_tsv_r8_repair05
```

Final-only evaluation is sufficient for frozen base, task-wise LoRAs, and joint IID unless needed for another diagnostic.

22. Headline metrics

Final overall performance:

$$OP = \frac{1}{8} \sum_{i=1}^8 A_{i,8}.$$

Forgetting:

$$F = \frac{1}{8} \sum_i (A_{i,i} - A_{i,8}).$$

Signed backward transfer:

$$BWT = \frac{1}{8} \sum_i (A_{i,8} - A_{i,i}).$$

Also calculate the clipped negative-only BWT used by the current TreeLoRA collector for compatibility.

Use distinct labels for these definitions.

23. Merge diagnostics

Every merge must save:

```
merge_id
level
child intervals
parent interval
task composition

merge_method
merge configuration hash
repair configuration hash

child update norms
child cosine similarity

input singular spectra
rank-8 retained energy
pre/post compression rank

Core-space basis dimensions
Core-space singular spectra
TSV diagnostics

pre-compression validation metrics
post-compression validation metrics
post-repair validation metrics

repair examples
repair tokens
repair steps
```

```
merge wall time  
repair wall time
```

Produce plots of:

```
merge damage vs level  
merge damage vs retained spectral energy  
within-task vs cross-task merge damage  
SVD merge vs Core+TSV  
0% vs 5% repair  
consolidation depth vs accumulated loss
```

24. Retrained-parent calibration

For several selected merges, train an expensive fresh rank-8 parent from the frozen base on the full source union.

At minimum:

```
one same-task L0→L1 merge  
one cross-task L0→L1 merge  
one L1→L2 merge  
one high-level merge
```

Compare:

```
fresh union-trained parent  
SVD merge  
SVD merge + repair  
Core+TSV  
Core+TSV + repair
```

These are calibration oracles, not deployed VAMP nodes.

25. Code organization

All TRACE-specific implementation code goes under its own package:

```
src/apm/continual/trace/
```

Recommended structure:

```
src/apm/continual/trace/  
    __init__.py  
  
    protocol.py  
    data.py  
    artifacts.py  
  
    leaf_training.py  
    baselines.py  
  
    bank.py  
    lineage.py  
  
    merging/  
        __init__.py  
        common.py  
        svd_mean.py  
        core_space.py  
        tsv.py  
        core_tsv.py  
  
    repair.py  
    reservoirs.py  
  
    routing.py  
    evaluation.py  
    metrics.py  
  
    scheduler.py  
    checkpoints.py  
    runtime_budget.py  
  
    reporting.py  
    notifications.py  
  
    runpod.py  
    cli.py
```

Tests go under:

```
tests/continual/trace/
```

Configuration goes under:

```
configs/trace/
```

RunPod packaging goes under:

```
docker/trace/  
scripts/trace/
```

Do not scatter TRACE experiment logic through unrelated `continual` modules.

Reuse generic existing VAMP components where appropriate, but TRACE-specific orchestration belongs inside `continual/trace`.

The branch already contains reusable continual-learning, routing, language-model, and LoRA infrastructure.

26. RunPod deployment target

Primary remote configuration:

```
1 RunPod Pod  
2 × RTX 4090  
persistent Network Volume
```

Use a **Network Volume**, not merely container storage.

RunPod documents Network Volumes as independent of the compute Pod: they remain after a Pod is terminated and can later be attached to another Pod.

Recommended initial storage:

```
150 GB network volume  
50 GB container disk
```

All important state must be under:

```
/workspace/vamp-trace/
```

because `/workspace` is the mounted persistent volume.

Container filesystem state is disposable.

27. Docker packaging

Provide:

```
docker/trace/Dockerfile
docker/trace/entrypoint.sh
```

The image should contain:

```
Ubuntu 22.04 or compatible
CUDA 12.x runtime
Python 3.11
PyTorch
Transformers
PEFT
Accelerate
safetensors
datasets
TRACE metric dependencies
matplotlib
pandas
pyarrow
pytest
Git
RunPod CLI/API support
```

Pin all material package versions.

HF caches must be redirected to the network volume:

```
HF_HOME=/workspace/vamp-trace/cache/huggingface
HF_DATASETS_CACHE=/workspace/vamp-trace/cache/huggingface/datasets
TRANSFORMERS_CACHE=/workspace/vamp-trace/cache/huggingface/models
```

Do not store the model cache solely in the ephemeral container layer.

28. Two-GPU scheduling

Do **not** data-parallelize a 1B model across both 4090s.

Use each 4090 as an independent worker.

A CPU coordinator owns the job DAG and launches subprocesses with:

```
CUDA_VISIBLE_DEVICES=0  
CUDA_VISIBLE_DEVICES=1
```

This gives much better utilization for this experiment because most units of work are independent.

Initial scheduling

After correctness smoke tests:

GPU 0

Begin the reference sequential LoRA baseline.

GPU 1

Begin immutable level-zero leaf training.

After the baseline completes, GPU 0 joins the leaf-training queue.

The 40 leaves are dynamically assigned to whichever GPU becomes available.

Once leaves exist

Parallelize:

```
GPU 0:  
  SVD merge tree  
  SVD repair tree  
  corresponding evaluations
```

```
GPU 1:
  Core+TSV merge tree
  Core+TSV repair tree
  corresponding evaluations
```

Baselines such as joint-IID and task-wise LoRAs are inserted whenever a GPU would otherwise be idle.

Evaluation jobs should be granular at approximately:

```
method × stage × TRACE task
```

so both GPUs remain busy.

29. Job DAG and resumability

Every expensive operation is a separately addressable job.

Examples:

```
train_leaf_017
train_joint_iid
train_seq_reference

merge_<policy>_<node>

repair_<policy>_<node>

eval_<policy>_<stage>_<task>
build_report
```

Each job has states:

```
PENDING
RUNNING
CHECKPOINTED
COMPLETE
FAILED
PAUSED
```

Completed jobs are never rerun unless explicitly forced.

A job's identity must include all semantically relevant inputs and configuration hashes.

30. Training checkpoints

Training jobs must checkpoint at least:

```
every 50 optimizer steps  
or  
every 2 minutes
```

whichever comes first.

Checkpoint:

```
LoRA weights  
optimizer state  
scheduler state  
RNG state  
epoch  
batch cursor  
data-order state  
gradient-accumulation position
```

Resume must continue from the saved batch rather than restarting the entire adapter.

31. Evaluation checkpoints

Generation/evaluation can be the longest part of the experiment.

After every generated example, or at least every small fixed batch, append output atomically.

Evaluation state contains:

```
method  
stage  
task  
next_example_index  
RNG state
```

```
generated outputs so far  
metric accumulator state
```

An interrupted 20Minuten or MeetingBank evaluation must resume at the unfinished example rather than regenerate the task.

32. 24-hour execution budget

The job must **not discard experimental conditions merely because the run is taking longer than expected.**

Instead implement a hard resumable pause policy.

Define:

```
soft_pause_request = 23h 30m  
hard_runtime_budget = 24h
```

measured from the beginning of the remote experiment session.

At the soft threshold:

1. stop dispatching new long-running jobs;
2. request all workers to checkpoint at the next safe boundary;
3. allow short already-running atomic work to finish;
4. persist the complete scheduler state.

Before the 24-hour threshold:

1. mark unfinished jobs **PAUSED** ;
2. generate an interim report;
3. send the configured notification;
4. flush and fsync critical manifests;
5. terminate the RunPod compute Pod while leaving the Network Volume intact.

RunPod Network Volumes persist independently after Pod termination. Pods with Network Volumes cannot use the ordinary stop/resume behavior, so the correct pause operation is to preserve state on the Network Volume and terminate/redeploy the compute Pod.

Never delete the Network Volume as part of pause or cleanup.

33. RunPod self-termination

The remote profile must receive:

```
RUNPOD_API_KEY  
RUNPOD_POD_ID
```

through secrets/environment variables.

Do not log either secret.

After successful checkpoint/report/notification at the 24-hour boundary, the supervisor may call the RunPod Pod termination API on its own Pod.

The volume ID is never passed to a delete operation.

If API-based termination is unavailable, the launcher must provide an independent watchdog capable of terminating the Pod after the supervisor has created:

```
state/SAFE_TO_TERMINATE
```

Never terminate merely because a timer fired without first establishing a durable resumable state.

34. Notification requirement

The remote run must support a notification hook.

Required environment interface:

```
VAMP_NOTIFY_WEBHOOK_URL
```

At startup perform a notification self-test.

Send notifications for:

```
experiment started  
experiment completed
```

```
experiment paused at 24h
experiment failed irrecoverably
```

The 24-hour notification must include:

```
run ID
elapsed time
completed conditions
unfinished conditions
current best provisional scores
interim report path
network volume identifier
exact resume command
estimated remaining runtime
```

Notification failure must not destroy or invalidate the run.

Record failed notification delivery in the persistent event log.

Do not claim notification is configured unless the startup self-test succeeds.

35. Interim report at 24 hours

Generate:

```
reports/INTERIM_<timestamp>.md
reports/INTERIM_<timestamp>.html
```

The report must clearly say:

```
PRELIMINARY – RUN PAUSED BEFORE COMPLETION
```

Include:

```
completed experiment matrix
available TRACE scores
merge diagnostics so far
repair results so far
routing results so far
GPU utilization
```

```
wall-clock breakdown
observed training throughput
observed evaluation throughput
remaining jobs
updated ETA
exact resume instructions
```

No partially evaluated method should be presented as final.

36. Resume behavior

Redeploy another 2×4090 Pod against the same Network Volume.

Run:

```
python -m apm.continual.trace.cli resume
--run /workspace/vamp-trace/runs/<run_id>
```

The coordinator:

1. validates all manifests and hashes;
2. discovers completed artifacts;
3. restores partially completed jobs;
4. reconstructs the DAG;
5. schedules only unfinished work.

No completed leaf, merge, baseline, or evaluation is recomputed.

37. Estimated completion time on 2×4090

This is an engineering estimate until the first calibration run produces measured throughput.

Each full leaf-equivalent training condition represents only about:

20,000

example presentations and roughly order- 10^7 sequence-token presentations given TRACE's task lengths and truncation.

The experiment is therefore expected to be **evaluation-bound rather than training-bound**.

For a well-optimized two-worker implementation, budget approximately:

container/model/data warmup	0.3–0.7 h
correctness/parity smoke	0.3–0.7 h
sequential + leaves + baselines	2–4 h
primary merge/repair hierarchies	0.5–1.5 h
TRACE task-boundary generation/eval	4–8 h
routing/oracle diagnostics	1–3 h
aggregation/reporting	0.2–0.5 h

Expected first complete run:

8–16 hours

Conservative allowance for slow HF generation, long MeetingBank examples, compilation/cache misses, and Core/repair sweeps:

16–22 hours

The hard operational policy remains:

24 hours $\Rightarrow$ checkpoint, interim report, notify, terminate Pod

rather than dropping work.

38. Runtime ETA calibration

Do not rely solely on the static estimate above.

During the first run measure:

training tokens/sec by sequence-length bucket
optimizer steps/sec
generation tokens/sec by task
prompt-prefill tokens/sec
merge time/node
repair time/example

After at least 15 minutes of representative work, continuously calculate ETA from actual remaining jobs.

Show:

static pre-run estimate

live measured ETA

confidence/range

in the status output.

The 24-hour supervisor uses actual elapsed time, not ETA, to decide when to pause.

39. Current comparison targets

Recent reported Llama-3.2-1B TRACE results provide approximate targets including a CRAFT result around **44.17 OP** and a task-wise isolated reference around **50.91**. These are useful comparison targets, but any final SOTA claim must be rechecked against current literature and protocol compatibility before publication.

The important internal comparisons remain:

VAMP – sequential LoRA

and:

joint IID LoRA – VAMP.

40. Required primary result table

The report must contain, at minimum:

Method	OP	Forgetting	BWT	Training presentations	Replay presentations	Final live LoRAs	Task-free
Frozen base				0	0	0	yes
Seq LoRA				20k	0	1	yes
Seq LoRA 40				20k	0	1	yes
Joint IID LoRA				20k	—	1	yes
Task-wise LoRA				20k	0	8	no
SVD log- <i>t</i>				20k	0	7	yes
SVD log- <i>t</i> + 5% repair				20k	measured	7	yes

Method	OP	Forgetting	BWT	Training presentations	Replay presentations	Final live LoRAs	Task-free
Core+TSV log- t				20k	0	7	yes
Core+TSV log- t + 5% repair				20k	measured	7	yes

Also report task-aware and answer-oracle routing as separate diagnostic columns/tables.

41. Required artifact-reuse experiment

After the primary run finishes, prove that the artifact architecture actually works.

Without retraining a single leaf:

1. change Core scaling;
2. change repair fraction;
3. rebuild the hierarchy;
4. reevaluate at least one final condition.

The report must show:

```
leaf training steps reused: 100%
leaf adapter hashes unchanged: true
new gradient work: repair only
```

This is a required system-level acceptance test.

42. Correctness tests

At minimum verify:

1. all 40 immutable leaves have distinct content hashes;
2. rerunning leaf-discovery does not retrain an existing valid leaf;
3. zero LoRA equals frozen-base logits;
4. PEFT save/load preserves logits;
5. exact stacked rank-16 sum equals the dense weighted child sum;
6. compact rank-8 SVD equals dense truncated SVD on test matrices;
7. Core-space reconstruction matches its dense reference implementation on small matrices;
8. Core+TSV compact rank-8 output matches dense Core+TSV followed by truncated SVD;

9. α/r scaling is correct;
10. the temporal topology is deterministic;
11. 40 arrivals produce exactly 33 merges and seven live nodes;
12. repair sample selection is deterministic;
13. changing repair fraction does not alter leaf hashes;
14. changing merge algorithm does not alter leaf hashes;
15. changing routing does not alter any trained adapter;
16. no headline router reads answer tokens;
17. test examples never enter training, repair, merger tuning, or router tuning;
18. job interruption/resume produces equivalent final adapters;
19. evaluation interruption resumes at the correct example;
20. Pod pause leaves all required state on `/workspace` ;
21. a resumed Pod reconstructs the exact remaining job DAG;
22. network-volume deletion is not present in any automatic cleanup path.

43. Resolved initial experiment matrix

Initial seed:

1234

Primary conditions:

```
seq_lora_reference
seq_lora_40
joint_iid_lora
taskwise_lora

vamp_svd_r8_repair000
vamp_svd_r8_repair005

vamp_core_tsv_r8_scale03_repair000
vamp_core_tsv_r8_scale03_repair005
```

Persist sufficient state to later generate, with no leaf retraining:

```
Core scale:
  0.1
  0.3
  0.5
  0.7
  0.9
```

Repair fraction:

0
0.02
0.05
0.10
0.25

Output rank:

8
optionally 12 / 16 as later diagnostics

44. Definition of done

The experiment is complete when:

- the reference TRACE control is reproduced;
- all 40 immutable level-zero adapters exist;
- all LoRA artifacts and lineage metadata remain permanently available;
- both SVD and Core+TSV temporal trees have been built;
- both zero-repair and 5%-repair versions have been evaluated;
- sequential, joint-IID, task-wise, and chunked-sequential controls exist;
- task-free, task-aware, and answer-oracle routing results exist;
- merge spectra and Core-Space diagnostics exist;
- training/replay/addressing/memory/wall-clock costs are accounted for;
- a no-leaf-retraining parameter rerun has been demonstrated;
- the entire job runs remotely on a 2×4090 RunPod;
- interruption and resume have been tested;
- the 24-hour budget automatically produces a durable interim report and notification rather than dropping experiment components.

The final report must answer:

Does log- t VAMP materially outperform sequential LoRA?

How close does it get to joint-IID LoRA?

Does Core+TSV beat simple rank-8 SVD merging?

How much replay is needed, if any?

Does merge error compound with temporal depth?

How much of the remaining error is addressing rather than storage?

and:

Can new merge/replay policies be tested without ever retraining the leaves?

That last property is part of the implementation, not merely an experimental convenience.